

COMP0496 – Programação A

Programação Orientada a Objetos — Parte 1

Classes, Objetos, Atributos e Métodos

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

Aquecimento — Tudo em Python é Objeto

O Problema — A Sopa de Variáveis

A Solução — Classes e Objetos

Atributos de Classe vs Instância

Prática — Classe Cardápio

Resumo

Aquecimento — Tudo em Python é Objeto

Você já usa objetos sem saber! Veja o que `type()` retorna:

```
1 >>> type("Oi")
2 <class 'str'>
3 >>> type([1, 2, 3])
4 <class 'list'>
5 >>> type(42)
6 <class 'int'>
7
```

Cada valor em Python pertence a uma **classe** — um tipo que define quais operações são possíveis.

Métodos são funções que “pertencem” a um objeto:

```
1  >>> "programacao".upper()  
2  'PROGRAMACAO'  
3  
4  >>> frutas = ["banana", "manga"]  
5  >>> frutas.append("uva")  
6  >>> frutas  
7  ['banana', 'manga', 'uva']  
8
```

Ponto-chave

POO = criar **seus próprios tipos** (classes) com seus próprios métodos e atributos.

O Problema — A Sopa de Variáveis

Você abriu um food truck e precisa de um sistema para gerenciar o cardápio.

Primeira tentativa: variáveis soltas.

```
1  lanche1_nome = "X-Tudo"
2  lanche1_preco = 18.50
3  lanche1_ingredientes = "pao, hamburguer, queijo, ovo, bacon"
4
5  lanche2_nome = "X-Salada"
6  lanche2_preco = 14.00
7  lanche2_ingredientes = "pao, hamburguer, queijo, alface, tomate"
8
```

- **Dados soltos:** nome, preço e ingredientes não estão conectados — qualquer variável pode ser alterada isoladamente
- **Não escala:** com 50 lanches, seriam 150 variáveis!
- **Erros de digitação:** `lanche1_preco` vs. `lanche1_prço` — sem aviso
- **Sem comportamento:** não há como pedir a um lanche que “se descreva”

Analogia

É como anotar pedidos em guardanapos separados: fácil perder, fácil trocar, impossível de organizar.

A Solução — Classes e Objetos

Classe = Receita

- Define *quais* informações um lanche tem
- Define *o que* um lanche pode fazer
- É um molde, um modelo
- Existe uma única vez no código

Objeto = Lanche Pronto

- Preenche a receita com dados reais
- Cada objeto é independente
- Pode haver muitos objetos da mesma classe
- Vive na memória do programa

```
1 class Lanche:
2     """Representa um lanche do cardapio."""
3
4     def __init__(self, nome, preco, ingredientes):
5         self.nome = nome
6         self.preco = preco
7         self.ingredientes = ingredientes
8
9     def descrever(self):
10         return f"{self.nome} - R${self.preco:.2f}"
11
```

- `__init__` é o **construtor** — é chamado automaticamente ao criar um objeto
- `self` é uma referência ao **próprio objeto** que está sendo criado
- `self.nome = nome` cria um **atributo de instância** — cada objeto terá o seu

Atenção

`self` **sempre** é o primeiro parâmetro de qualquer método, mas **não** é passado na chamada — Python cuida disso.

```
1  xtudo = Lanche("X-Tudo", 18.50, "pao, hamburguer, queijo, ovo, bacon")
2  xsalada = Lanche("X-Salada", 14.00, "pao, hamburguer, queijo, alface")
3
4  print(xtudo.descrever())      # X-Tudo - R$18.50
5  print(xsalada.nome)          # X-Salada
6  print(xsalada.preco)         # 14.0
7
```

Cada objeto é independente: alterar `xtudo.preco` não afeta `xsalada.preco`.

Sem OOP

```
1  lanche1_nome = "X-Tudo"
2  lanche1_preco = 18.50
3  # ... e mais 148 variaveis
4
```

Com OOP

```
1  xtudo = Lanche("X-Tudo", 18.50,
2              "pao, hamburger...")
3  # 1 variavel = 1 lanche completo
4
```

Dados e comportamento ficam **juntos** — é isso que “orientação a objetos” significa.

Conceito	Significado
Classe	Modelo/receita que define atributos e métodos
Objeto	Instância concreta de uma classe
<code>__init__</code>	Método construtor — inicializa o objeto
<code>self</code>	Referência ao próprio objeto
Atributo	Variável que pertence ao objeto
Método	Função que pertence à classe

```
1  class Lanche:
2      def __init__(self, nome, preco, ingredientes):
3          self.nome = nome
4          self.preco = preco
5          self.ingredientes = ingredientes
6
7      def descrever(self):
8          return f"{self.nome} - R${self.preco:.2f}"
9
10     def aplicar_desconto(self, percentual):
11         self.preco *= (1 - percentual / 100)
12
13     def tem_ingrediente(self, ingrediente):
14         return ingrediente in self.ingredientes
15
```



```
1  xtudo = Lanche("X-Tudo", 18.50, "pao, hamburguer, queijo, ovo")
2
3  print(xtudo.descrever())           # X-Tudo - R$18.50
4
5  xtudo.aplicar_desconto(10)
6  print(xtudo.descrever())           # X-Tudo - R$16.65
7
8  print(xtudo.tem_ingrediente("ovo")) # True
9  print(xtudo.tem_ingrediente("bacon")) # False
10
```

O objeto **sabe** como se descrever, como aplicar desconto e como verificar ingredientes — o comportamento está *dentro* do objeto.

Atributos de Classe vs Instância

	Atributo de Classe	Atributo de Instância
Onde define	Dentro da classe, fora de métodos	Dentro de <code>__init__</code>
Compartilhado	Sim, por todos os objetos	Não, cada objeto tem o seu
Acesso	<code>Classe.atributo</code>	<code>self.atributo</code>
Exemplo	Taxa de serviço (10%)	Nome do lanche

Exemplo: Atributo de Classe



```
1 class Lanche:
2     taxa_servico = 0.10  # compartilhado por TODOS os lanches
3
4     def __init__(self, nome, preco, ingredientes):
5         self.nome = nome
6         self.preco = preco
7         self.ingredientes = ingredientes
8
9     def preco_com_taxa(self):
10         return self.preco * (1 + Lanche.taxa_servico)
11
```

```
1 xtudo = Lanche("X-Tudo", 18.50, "pao, hamburguer, queijo")
2 print(xtudo.preco_com_taxa())  # 20.35
3
```

```
1 class Lanche:
2     extras = [] # PERIGO! Lista compartilhada
3
4     def __init__(self, nome):
5         self.nome = nome
6
7     def adicionar_extra(self, extra):
8         self.extras.append(extra)
9
```

```
1 a = Lanche("X-Tudo")
2 b = Lanche("X-Salada")
3 a.adicionar_extra("bacon")
4 print(b.extras) # ['bacon'] -- b tambem tem bacon?!
5
```

```
1 class Lanche:
2     def __init__(self, nome):
3         self.nome = nome
4         self.extras = [] # CORRETO: cada objeto tem sua lista
5
6     def adicionar_extra(self, extra):
7         self.extras.append(extra)
8
```

```
1 a = Lanche("X-Tudo")
2 b = Lanche("X-Salada")
3 a.adicionar_extra("bacon")
4 print(a.extras) # ['bacon']
5 print(b.extras) # [] -- independente!
6
```

Prática — Classe Cardápio

Um Cardápio contém uma **lista de objetos** Lanche.

Isso é chamado de **composição**: um objeto é composto por outros objetos.

Analogia

O cardápio do food truck não é um lanche — ele *contém* lanches. A relação é “tem-um” (has-a), não “é-um” (is-a).


```
1 class Cardapio:
2     """Gerencia a colecao de lanches do food truck."""
3
4     def __init__(self, nome_food_truck):
5         self.nome_food_truck = nome_food_truck
6         self.lanches = [] # lista de objetos Lanche
7
8     def adicionar(self, lanche):
9         self.lanches.append(lanche)
10
11     def mostrar(self):
12         print(f"=== Cardapio: {self.nome_food_truck} ===")
13         for lanche in self.lanches:
14             print(f"    {lanche.descrever()}")
15
```

```
1  class Cardapio:
2      # ... (continuacao)
3
4      def preco_medio(self):
5          if not self.lanches:
6              return 0
7          total = sum(l.preco for l in self.lanches)
8          return total / len(self.lanches)
9
10     def mais_carro(self):
11         if not self.lanches:
12             return None
13         return max(self.lanches, key=lambda l: l.preco)
14
```

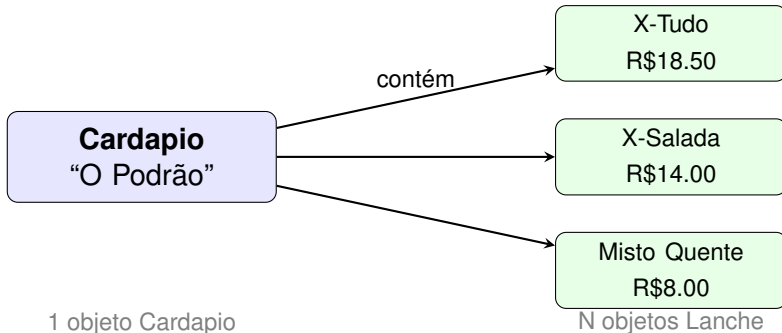
```
1 cardapio = Cardapio("0 Podrao")
2 cardapio.adicionar(Lanche("X-Tudo", 18.50, "pao, hamburger..."))
3 cardapio.adicionar(Lanche("X-Salada", 14.00, "pao, alface..."))
4 cardapio.adicionar(Lanche("Misto Quente", 8.00, "pao, presunto"))
5
6 cardapio.mostrar()
7
```

Saída:

```
=== Cardapio: 0 Podrao ===
X-Tudo - R$18.50
X-Salada - R$14.00
Misto Quente - R$8.00
```

```
1 print(f"Preco medio: R${cardapio.preco_medio():.2f}")
2 # Preco medio: R$13.50
3
4 campeão = cardapio.mais_caro()
5 print(f"Mais caro: {campeao.descrever()}")
6 # Mais caro: X-Tudo - R$18.50
7
```

Note como `mais_caro()` retorna um **objeto** `Lanche`, e podemos chamar `descrever()` nele — objetos conversam entre si!



Resumo

Conceito	O que aprendemos
Classe	Modelo que define atributos e métodos
Objeto	Instância concreta com dados próprios
<code>__init__</code>	Construtor — inicializa cada objeto
<code>self</code>	Referência ao próprio objeto
Atrib. de instância	Pertence ao objeto (criado em <code>__init__</code>)
Atrib. de classe	Compartilhado por todos os objetos
Método	Função que opera sobre o objeto
Composição	Objeto contém outros objetos (“tem-um”)

1. **Esquecer** `self` no primeiro parâmetro de métodos → `TypeError`
2. **Listas/dicts como atributo de classe** → compartilhamento indesejado entre objetos
3. **Confundir classe com objeto** → `Lanche.descrever()` sem instância dá erro
4. **Não chamar `__init__` corretamente** → atributos faltando (`AttributeError`)

- **Encapsulamento:** atributos públicos vs. privados
- `@property`: getters e setters pythônicos
- **Métodos especiais:** `__str__`, `__repr__`, `__eq__`
- **Validação de dados:** garantir que preço > 0 , nome não vazio

O food truck “O Podrão” continua evoluindo!

Dúvidas?

Prof. Dr. André Yoshiaki Kashiwabara

`andreyoshiaki@dcomp.ufs.br`

DCOMP — Universidade Federal de Sergipe